IT4931 / IT5384 / IT4043E · SOICT · HUST

Apache Spark

Lecture 10 — Batch data processing

RDD · DAG / stages · DataFrame · Spark SQL

Learning outcomes (CLO2, CLO3)

1. **Describe** Spark vs MapReduce; Driver / Executors
2. **Explain** RDD: partitions, lazy **transformations** vs **actions**
3. **Trace** jobs → stages → tasks; lineage for fault tolerance
4. **Use** DataFrame / Spark SQL as the modern analytics API

Roadmap

1. **10.1** Introduction — why Spark
2. **10.2** RDD model
3. **10.3** DAG, stages, execution
4. **10.4** DataFrame & Spark SQL

10.1 Introduction

MapReduce pain: iterative I/O



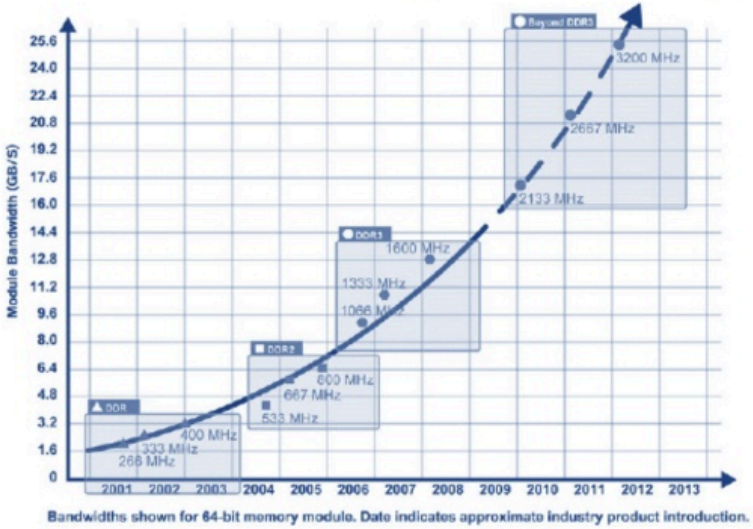
I/O hierarchy reality

Layer	Order of magnitude
CPU / DRAM	~10 GB/s · ~0.1 ms · expensive
Local SSD / HDD	~100–600 MB/s · ms latency · cheap
Network (same rack)	~1 Gb/s (~125 MB/s)
Cross-rack / cluster	often slower · shared

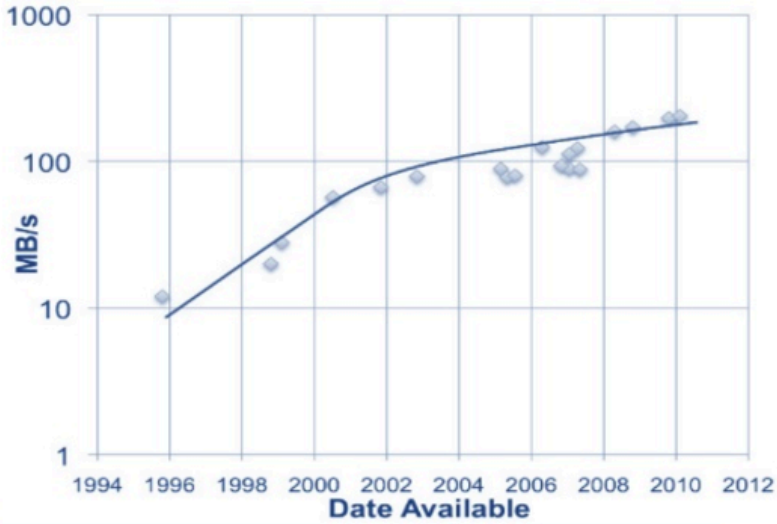
Disk & network dominate iterative MapReduce; **keep hot data in memory** when you can.

Memory can beat disk for iteration

- RAM throughput increasing **exponentially**



- Disk throughput increasing **slowly**



Memory-locality key to interactive response times

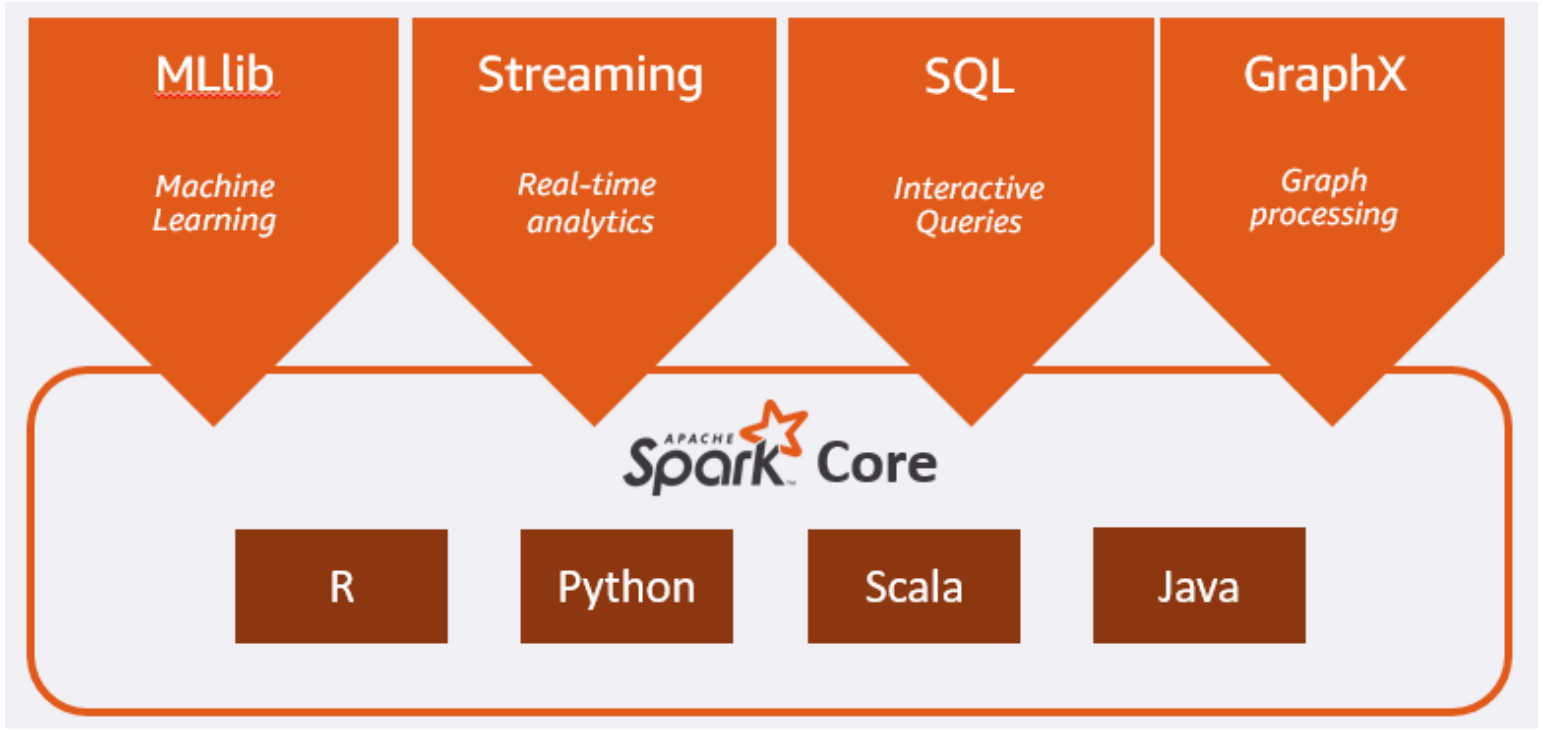
Unified analytics platform

Goals vs MapReduce-era stacks:

- **Iterative** algorithms (keep working sets hot)
- **Interactive** mining / exploration
- Fault tolerance · data locality · scale-out
- Hide distributed complexity behind a programming model

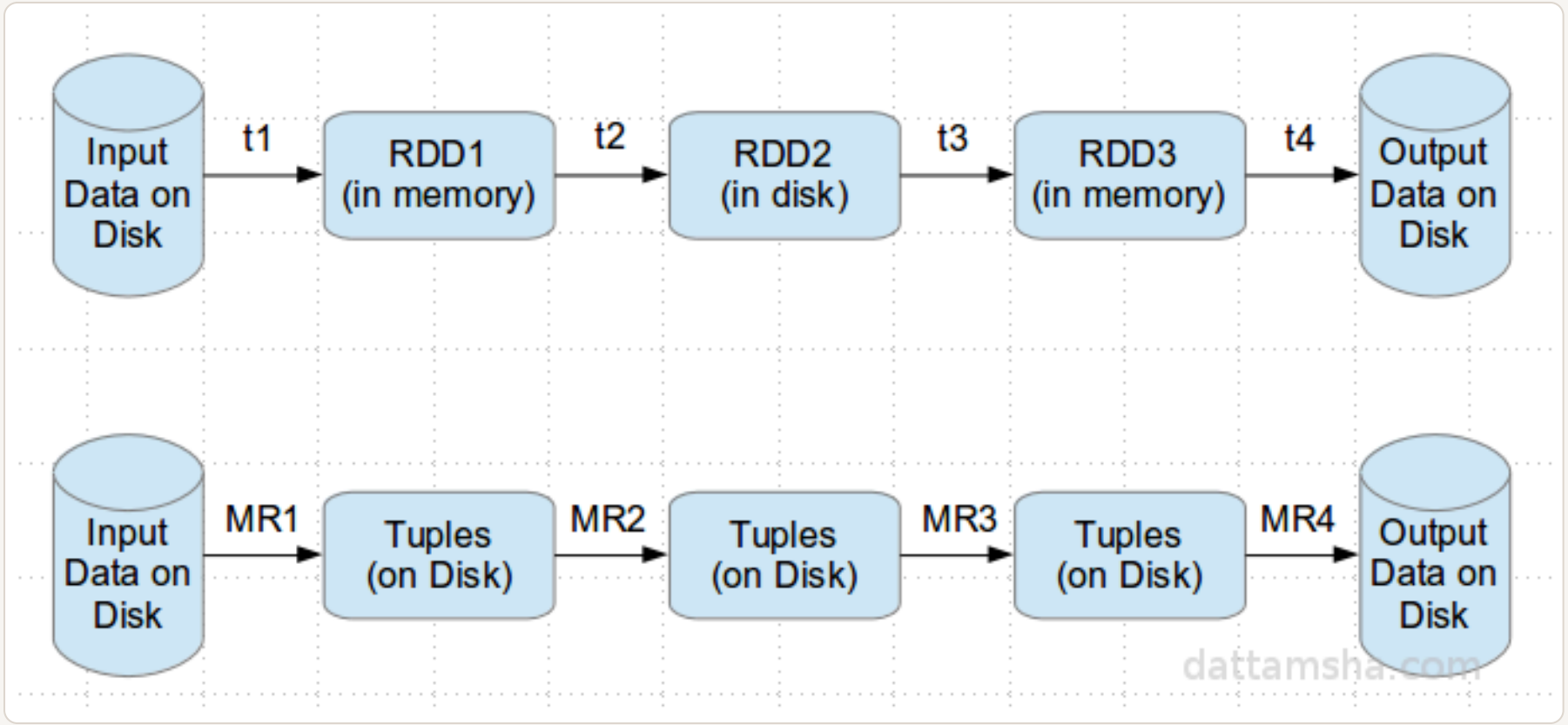
One engine for batch · interactive · streaming · ML.

Spark stack



Libraries on **Spark Core** · R / Python / Scala / Java.

In-memory vs HDFS round-trips

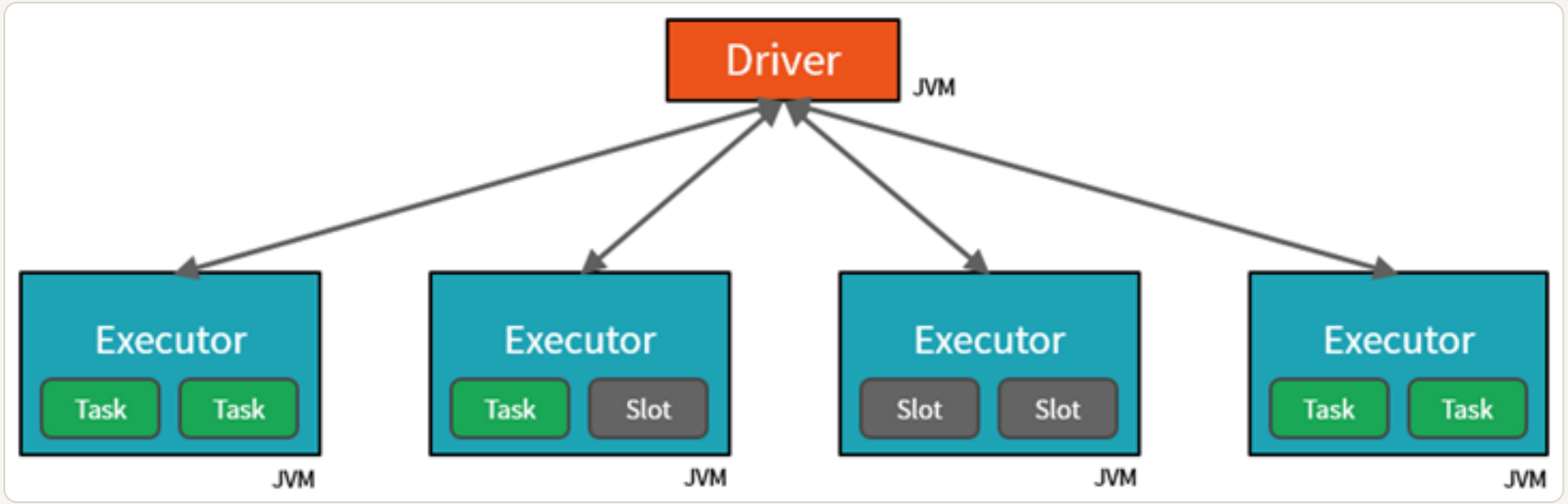


Spark vs MapReduce

	Hadoop MR	Spark
Storage path	Mostly disk between stages	Memory + disk
Ops	Map + Reduce	Rich transforms + actions
Workloads	Batch	Batch, interactive, streaming
Languages	Java-centric historically	Scala, Java, Python, R, SQL

Classic sort benchmarks favored Spark via **memory + DAG** vs MR disk I/O (illustrative, not a 2026 SLA).

Driver, executors & cluster



Driver builds the plan · **executors** run tasks in slots · managers: standalone · YARN · K8s.

Where Spark runs (`SparkContext` / session)

Master	Meaning
`local` / `local[K]`	Single machine · 1 or *K* threads (dev)
`spark://HOST:PORT`	Spark standalone cluster
`yarn`	Hadoop YARN
`k8s://…`	Kubernetes (modern default on cloud)

First object in a program: **SparkSession** (creates the context that talks to the cluster).

10.2 Resilient Distributed Datasets

What is an RDD?

- Fault-tolerant, **partitioned** parallel collection
- Control **persistence** (memory/disk) & **partitioning**
- Rich **coarse-grained** operators (`map` , `filter` , `join` , …)
- Rebuild lost partitions via **lineage** (not fine-grained updates)

Creating RDDs

```python
# 1) parallelize a driver collection (small / experiments)
words = sc.parallelize(["fish", "cats", "dogs"])

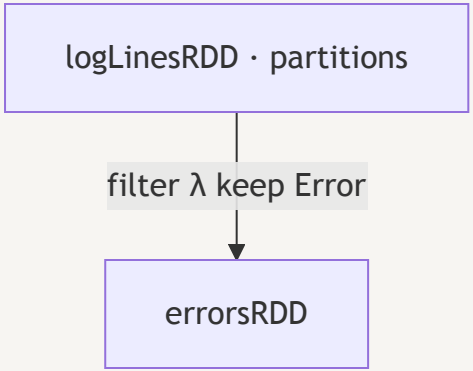
# 2) read external storage (HDFS, S3, local, …)
lines = sc.textFile("/path/to/README.md")
```

Same idea in Scala/Java; production jobs almost always start from **external** data.

Transformations vs actions

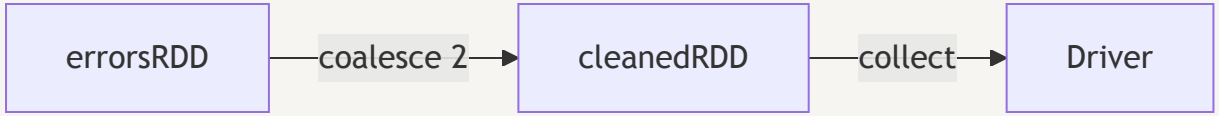
	Transformations	Actions
Timing	**Lazy** — build lineage	Trigger computation
Result	New RDD / DataFrame	Values to driver / side effects
Examples	`map` , `filter` , `join`	`collect` , `count` , `saveAsTextFile`

Example — filter transformation



Narrow transform: each partition filtered independently (some may become empty).

Example — collect action



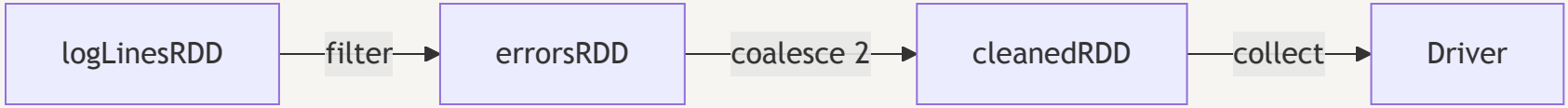
Action pulls results to the **driver** — keep datasets small or use distributed sinks.

Programming workflow (RDD)

1. Create RDD from external data / collection
2. Lazily transform → intermediate RDDs
3. `cache()` / `persist()` for reuse
4. Call **actions** to run & get results

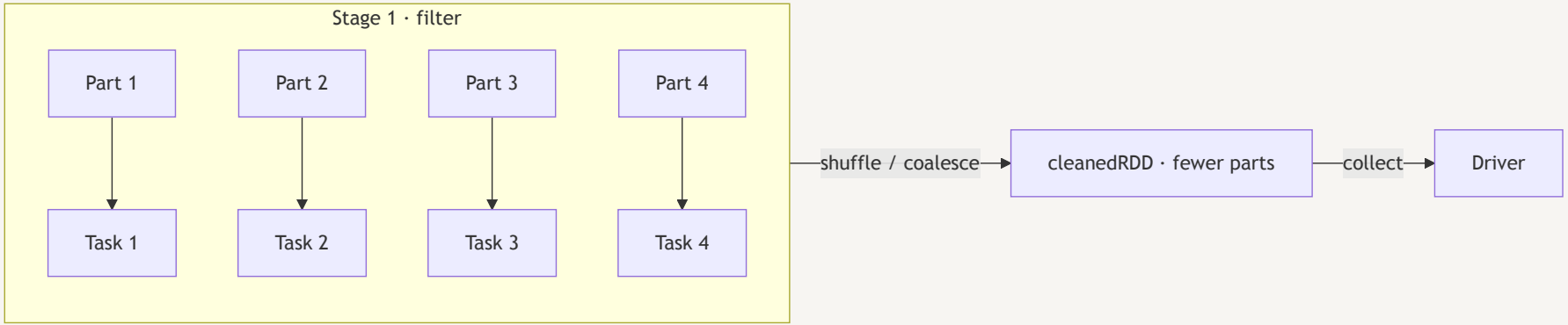
10.3 DAG & execution

Logical lineage (action triggers DAG)



- Transforms are **lazy** — Spark only records lineage
- An **action** (`collect` , `count` , …) schedules stages and **runs** the DAG
- The same lineage graph is how Spark **rebuilds** lost partitions later

Physical execution



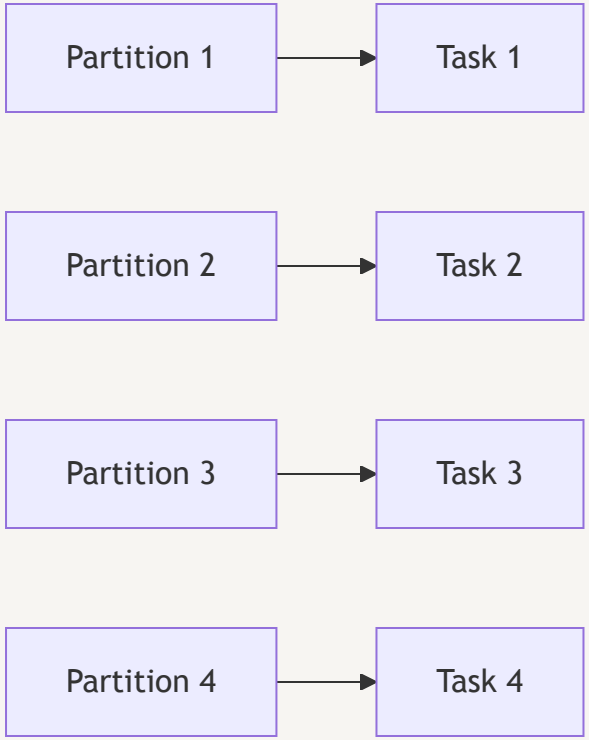
Physical plan = **tasks on executors** materializing each stage.

Multiple actions & caching

Without `cache`	**With** `cache` / `persist`
Each action may **recompute** the full lineage	Materialize once in memory (or disk)
`collect` then `count` → two full passes	Later actions **reuse** cached RDD
OK for one-shot jobs	Needed when the same RDD is used many times

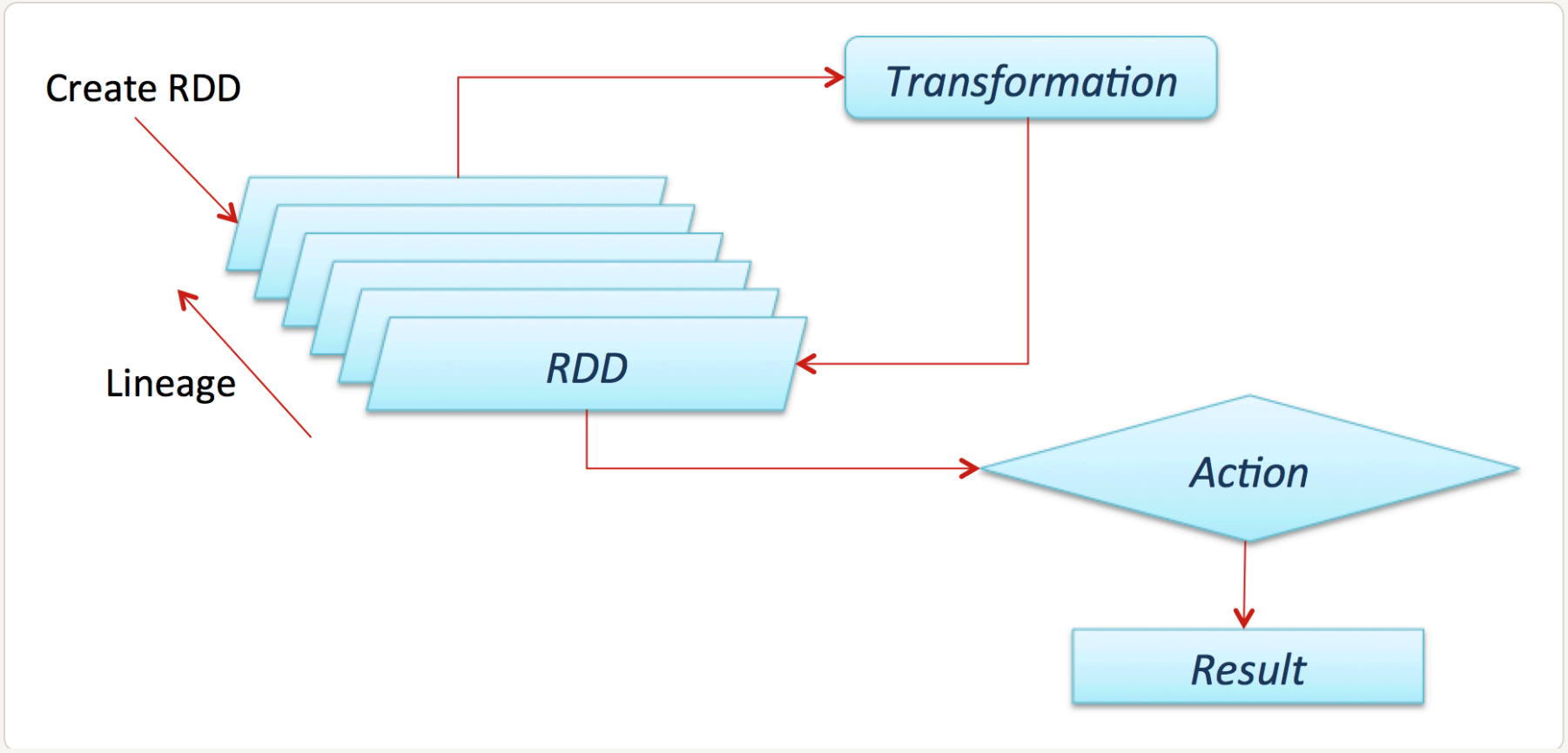
```
errors = logLines.filter(...).cache()
errors.count()          # computes + caches
errors.saveAsTextFile() # reuses cache
```

Partition → task

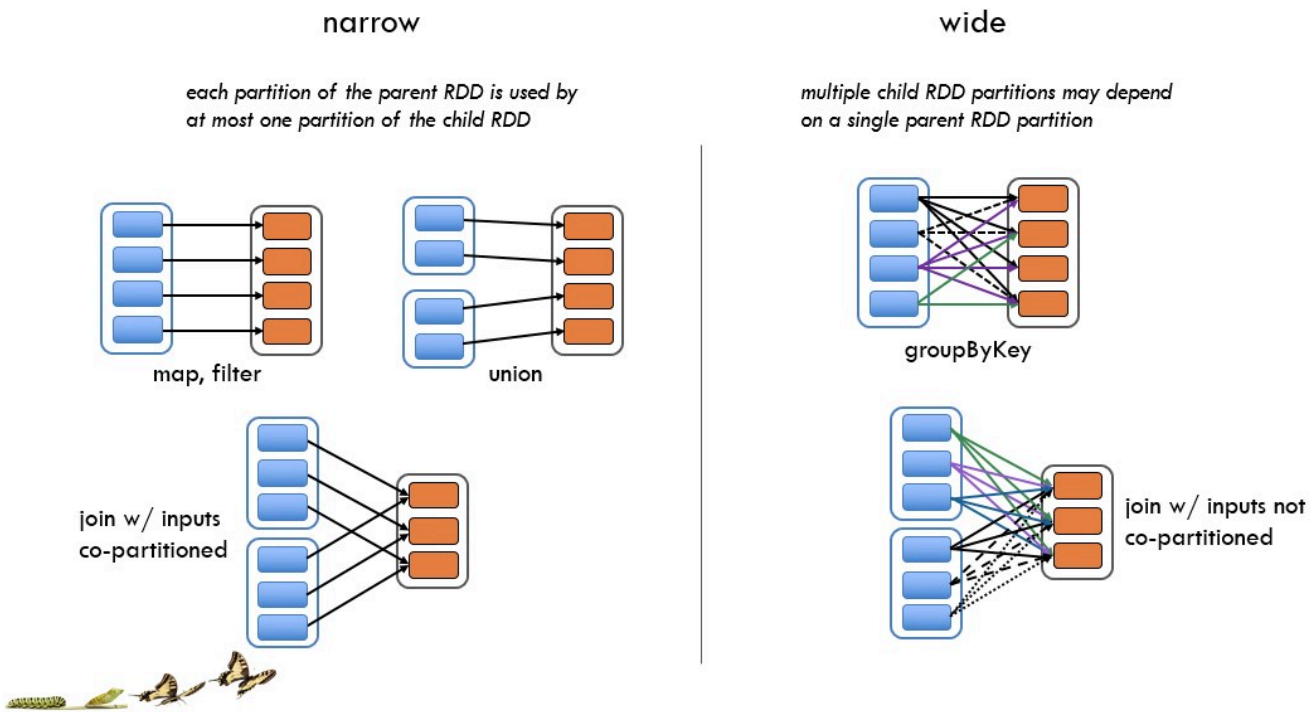


One **task** per partition **per stage** (until `coalesce` / `repartition` changes the count).

Fault tolerance via lineage

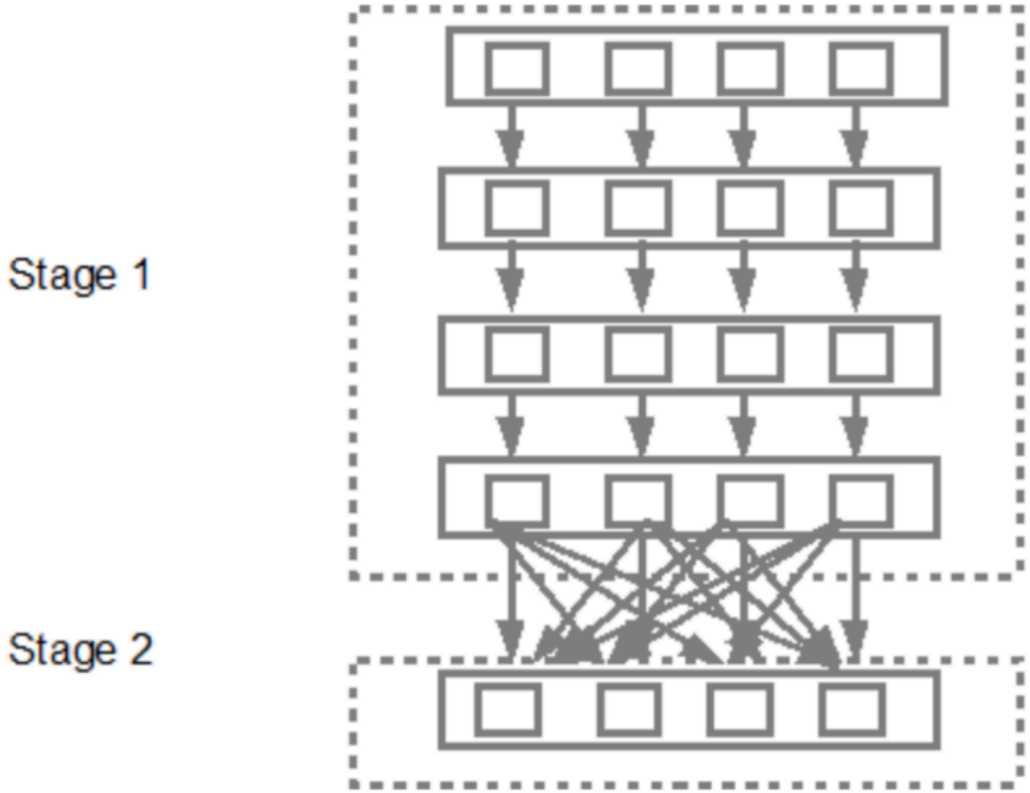


Narrow vs wide dependencies

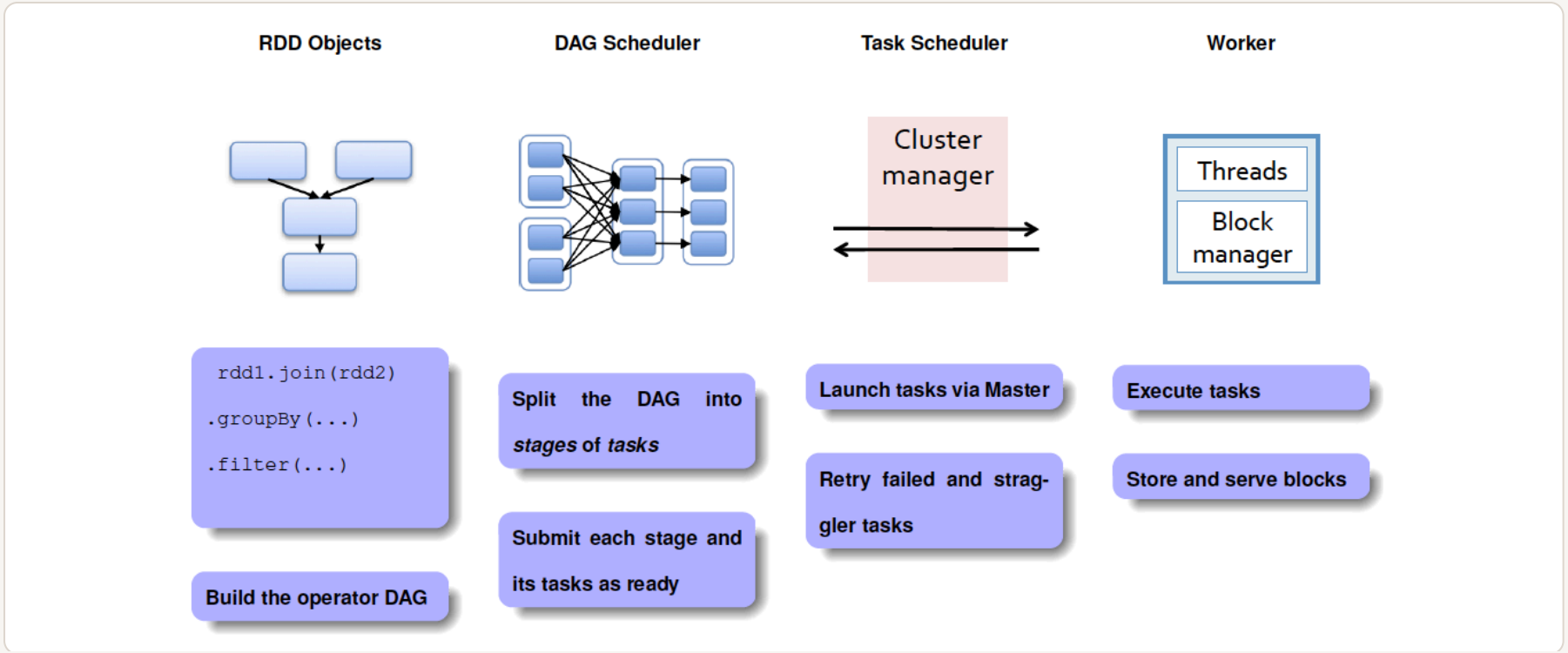


Narrow (map/filter): no shuffle · **Wide** (reduceByKey/join): shuffle → stage boundary.

Stages = pipelined narrow ops



Job lifecycle



Job (from an action) → **stages** → **tasks** on executors.

10.4 DataFrame & Spark SQL

DataFrame as primary API (Spark 2+)

- Immutable tabular abstraction with **schema**
- Same lineage / lazy model as RDD
- Catalyst optimizer + Tungsten execution
- Create from collections, files, Hive, JDBC, pandas, …

Prefer DataFrame/Dataset for analytics; RDD when you need low-level control.

Creating DataFrames

```
data = [("Alice", 1), ("Bob", 2), ("Bob", 2)]
df1 = spark.createDataFrame(data, ["name", "age"])

df_file = spark.read.parquet("s3://…/orders")
# also: .csv / .json / JDBC / Hive table / pandas → Spark
```

Same two paths as RDD: **driver collection** or **external source**.

DataFrame transformations (lazy)

API	Effect
`select(*cols)`	Project columns
`drop(col)`	Remove column(s)
`filter / where`	Keep rows matching predicate
`distinct()`	Drop duplicate rows
`sort / orderBy`	Order rows
`groupBy(…).agg(…)`	Aggregations

Build a new DataFrame — nothing runs until an **action**.

DataFrame actions

API	Effect
`show(n)`	Print first *n* rows
`take(n)` / `collect()`	Rows to driver (careful with `collect`)
`count()`	Number of rows
`describe(*cols)`	Quick numeric stats
`write.…`	Persist results

```
df2 = df1.distinct().sort("age", ascending=False)
df2.show()
df2.count()  # triggers the plan
```

DF workflow mirrors RDD

1. Create DataFrames
2. Lazily transform (`select` , `filter` , `groupBy` , …)
3. `cache()` when reused
4. Actions: `show` , `collect` , `count` , `write` , …

```
df = spark.read.parquet("s3://…/orders")
df.filter(df.amount > 100).groupBy("country").count().show()
```

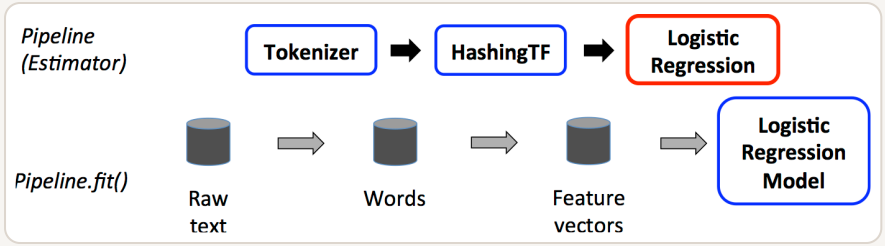
Spark SQL

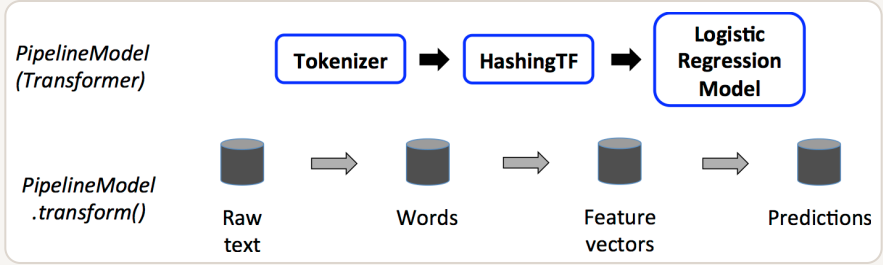
```sql
SELECT country, COUNT(*) AS n
FROM orders
WHERE amount > 100
GROUP BY country;
```

- Register temp views / use catalogs
- Same engine as DataFrame API
- Bridge to lakes (Parquet/ORC/Iceberg) & warehouses

MLlib glimpse → L13

Idea	In one line
Prefer `spark.ml`	DataFrame Pipelines (not RDD `mllib`)
Transformer	`transform(df)`
Estimator	`fit(df)` → Model (also a Transformer)
Pipeline	Chain stages · `fit` → **PipelineModel**





Full treatment (features, evaluators, CrossValidator, demo): **Lecture 13**.

Takeaways

1. Spark = unified engine; **memory + DAG** beat MR for iteration/interactive
2. **RDD**: partitioned, lazy transforms, actions trigger work
3. **Stages** split at shuffles; **lineage** rebuilds lost data
4. **DataFrame/SQL** = default analytics surface in 2026

Next

L11 — Spark Structured Streaming